

Polars for Data Consulting

API reference, analytics workflows, statistics, and charting patterns for autonomous code generation

PrevAI Health

August 2026

How to Use This Document

Table of Contents

1. Core Concepts
2. Installation and Environment
3. Reading and Writing Data (I/O)
 - 3.1 Eager readers (`pl.read_*`) vs lazy scanners (`pl.scan_*`)
 - 3.2 CSV specifics
 - 3.3 Parquet (preferred for anything beyond quick scripts)
 - 3.4 Excel
 - 3.5 Databases
 - 3.6 JSON / NDJSON
 - 3.7 In-memory construction
4. Selecting, Filtering, and Sorting
 - 4.1 Column selection
 - 4.2 Row filtering
 - 4.3 Sorting
5. Expressions Deep Dive
 - 5.1 Building blocks
 - 5.2 Arithmetic and comparisons work directly on expressions
 - 5.3 `when / then / otherwise` — polars' `if/else`
 - 5.4 Applying an expression to many columns at once
 - 5.5 Aggregation expressions (used inside `.agg()` or `select` on a whole `df`)
 - 5.6 Combining multiple expressions cleanly
6. Transformations: `with_columns`, casting, `struct/list`
 - 6.1 Adding and replacing columns
 - 6.2 Casting types
 - 6.3 Renaming, reordering, dropping
 - 6.4 Structs (nested columns) and Lists
7. String, Date/Time, and List Operations
 - 7.1 String operations (`.str`)
 - 7.2 Date/time operations (`.dt`)
 - 7.3 List operations (`.list`)
8. Aggregations and `group_by`
 - 8.1 Basic `group_by` / `agg`
 - 8.2 Multiple aggregations, and getting a list of rows per group
 - 8.3 `group_by_dynamic` — time-bucketed aggregation (downsampling)
 - 8.4 Rolling aggregations over time (`rolling` / `group_by_rolling` equivalent via `.over()` + window)
 - 8.5 Pivoting an aggregation into wide format directly
9. Window Functions (`.over()`)
10. Joins, Concatenation, and Reshaping
 - 10.1 Joins
 - 10.2 Concatenation
 - 10.3 Reshaping: `pivot` (long -> wide) and `unpivot` (wide -> long)
11. Missing Data Handling
12. Performance: Lazy Execution, Streaming, Query Plans
 - 12.1 Always prefer lazy for pipelines
 - 12.2 Streaming for datasets bigger than RAM
 - 12.3 Query plan inspection
 - 12.4 General performance habits

- 13. Statistical Analysis with Polars
 - 13.1 Descriptive statistics
 - 13.2 Correlation
 - 13.3 Outlier detection
 - 13.4 Trend and period-over-period analysis
 - 13.5 Hypothesis testing and regression (hand off to scipy/statsmodels/numpy)
 - 13.6 Cohort / retention analysis pattern
- 14. Data Consulting Workflows
 - 14.1 Workflow: Profile a new/unfamiliar dataset
 - 14.2 Workflow: Diagnose a metric drop / “why did X change”
 - 14.3 Workflow: Segment / cohort comparison
 - 14.4 Workflow: Build a KPI / executive summary table
 - 14.5 Workflow: Data quality / anomaly sweep before trusting a number
 - 14.6 How to present results (report structure)
- 15. Charting and Visualization
 - 15.1 Quick native charts: `df.plot.*` (Altair backend)
 - 15.2 Matplotlib (best for static, print-ready report charts)
 - 15.3 Plotly (best for interactive dashboards / HTML deliverables)
 - 15.4 Chart-type selection guide
- 16. End-to-End Worked Case Study
- 17. Common Pitfalls and Anti-Patterns
- 18. Quick-Reference Cheat Sheet

How to Use This Document

You are a coding assistant generating Python data-analysis code with the **polars** library. You have no internet access, so this document is your source of truth for polars syntax, idioms, and workflow patterns. Treat every code block here as a verified, runnable pattern for **polars 1.44 (stable, August 2026)**. When a user asks for analysis, look up the closest matching pattern below, adapt column names and logic, and compose a solution from these building blocks rather than guessing at API names.

Ground rules to follow when generating code:

1. Prefer the **lazy API** (`pl.scan_csv`, `pl.scan_parquet`, `.lazy()` and `LazyFrame`) for anything beyond a quick look, and call `.collect()` only at the end of a pipeline.
2. Use **expressions** (`pl.col(...)`) inside `select`, `with_columns`, `filter`, `group_by().agg()` rather than looping over rows in Python. Row-wise Python loops over a `DataFrame` are almost always wrong in polars.
3. Method names are **snake_case**: `group_by`, `is_null`, `str.to_uppercase`, `dt.year`. There is no `groupby` (deprecated/removed) or camelCase in modern polars.
4. Never use `apply / map_elements` unless there is truly no expression-based way to do something — it is slow and breaks query optimization. Always look for a native expression first.
5. When producing a chart, default to `DataFrame.plot.<kind>()` (Altair backend) for quick interactive charts, or `matplotlib / plotly` for full control and static report output. See the Charting section.
6. When asked for “a data consulting analysis” or “a report,” follow the workflow templates in the Data Consulting Workflows section: profile the data first, state assumptions, then analyze, then summarize findings in business language, not just code output.

Table of Contents

- 1. Core Concepts
- 2. Installation and Environment
- 3. Reading and Writing Data (I/O)
- 4. Selecting, Filtering, and Sorting
- 5. Expressions Deep Dive
- 6. Transformations: with_columns, when/then/otherwise, casting
- 7. String, Date/Time, and List Operations
- 8. Aggregations and group_by
- 9. Window Functions (.over())
- 10. Joins, Concatenation, and Reshaping
- 11. Missing Data Handling
- 12. Performance: Lazy Execution, Streaming, Query Plans
- 13. Statistical Analysis with Polars
- 14. Data Consulting Workflows
- 15. Charting and Visualization
- 16. End-to-End Worked Case Study
- 17. Common Pitfalls and Anti-Patterns
- 18. Quick-Reference Cheat Sheet

1. Core Concepts

Polars has two core data structures and two execution modes.

Series — a single typed column, like `pl.Series("age", [23, 45, 12])`.

DataFrame — an eager, in-memory table of Series, similar to a pandas DataFrame but column-oriented (Apache Arrow backed) and multi-threaded by default.

LazyFrame — a *query plan*, not data. Operations on a LazyFrame (`filter`, `select`, `group_by`, `join`, ...) build up a graph of transformations that is only optimized and executed when you call `.collect()`, `.fetch()` (deprecated, use `.collect().head()` or `.collect(streaming=True)`), or a sink method. This is the recommended way to work with polars because the query optimizer can push down filters/projections, eliminate unused columns, and stream data larger than RAM.

```
import polars as pl

# Eager: runs immediately, loads everything into memory
df = pl.read_csv("sales.csv")

# Lazy: builds a plan, nothing executes yet
lf = pl.scan_csv("sales.csv")
result = (
    lf.filter(pl.col("region") == "EMEA")
      .group_by("product")
      .agg(pl.col("revenue").sum())
      .sort("revenue", descending=True)
)
df = result.collect()           # executes the optimized plan
print(result.explain())        # inspect the query plan without running it
```

Expressions (`pl.Expr`) are the core abstraction: a lazily-evaluated description of a column computation, e.g. `pl.col("price") * pl.col("qty")`. Expressions compose, can be reused across contexts, and run in parallel across columns and row-chunks. Nearly everything you do in polars is “build expressions, feed them into a context.”

Contexts are the methods that *consume* expressions:

Context	Purpose
<code>select(*exprs)</code>	Choose/compute columns, drop the rest
<code>with_columns(*exprs)</code>	Add or replace columns, keep the rest
<code>filter(*exprs)</code>	Keep rows where boolean expression(s) are true
<code>group_by(*keys).agg(*exprs)</code>	Aggregate per group
<code>sort(*keys)</code>	Order rows

```
df.select(pl.col("name"), (pl.col("price") * pl.col("qty")).alias("total"))
df.with_columns((pl.col("price") * pl.col("qty")).alias("total"))
df.filter(pl.col("total") > 100)
df.group_by("category").agg(pl.col("total").sum().alias("category_total"))
```

2. Installation and Environment

```
pip install polars                # CPU, auto-detects best SIMD build
pip install "polars[all]"         # + pandas/numpy interop, plotting, Excel, DB, cloud I/O extras
pip install polars-lts-cpu       # for older CPUs lacking AVX2, if "polars" segfaults
```

Common optional extras and what they unlock:

```
pip install polars[pandas]       # to_pandas() / from_pandas()
pip install polars[numpy]        # to_numpy() efficient paths
pip install polars[pyarrow]      # Arrow interop, some I/O formats
pip install polars[fsspec]       # remote filesystem scanning (s3://, gcs://, ...)
pip install polars[xlsx2csv]     # read_excel
pip install polars[deltalake]    # Delta Lake tables
pip install polars[sqlalchemy,connectorx] # read_database
pip install altair               # required for df.plot.*()
pip install matplotlib          # for matplotlib-based charts
```

Check your installed version and build info:

```
import polars as pl
print(pl.__version__)
print(pl.show_versions())
```

Useful global configuration for readable console output during analysis:

```
pl.Config.set_tbl_rows(50)
pl.Config.set_tbl_cols(30)
pl.Config.set_fmt_str_lengths(80)
pl.Config.set_tbl_width_chars(120)
```

3. Reading and Writing Data (I/O)

3.1 Eager readers (`pl.read_*`) vs lazy scanners (`pl.scan_*`)

Use `scan_*` whenever the file might be large, when you only need a subset of columns/rows, or when you're building a pipeline that will filter/aggregate before materializing. `scan_*` returns a `LazyFrame` and defers all I/O until `.collect()`.

```
# Eager — loads entire file into memory immediately
df = pl.read_csv("data.csv")
df = pl.read_parquet("data.parquet")
df = pl.read_json("data.json")
df = pl.read_ndjson("data.ndjson")
df = pl.read_excel("data.xlsx", sheet_name="Sheet1")

# Lazy — builds a query plan; nothing loaded until .collect()
lf = pl.scan_csv("data.csv")
lf = pl.scan_parquet("data*.parquet")          # glob patterns supported
lf = pl.scan_ndjson("events.ndjson")
lf = pl.scan_parquet("s3://bucket/path/*.parquet") # cloud paths (needs fsspec/object_store)

# Only pull what you need — the optimizer prunes columns/rows before reading
result = (
    pl.scan_parquet("big_dataset.parquet")
    .filter(pl.col("event_date") >= pl.date(2026, 1, 1))
    .select("user_id", "event_type", "revenue")
    .collect()
)
```

3.2 CSV specifics

```
df = pl.read_csv(
    "data.csv",
    separator=";",
    has_header=True,
    try_parse_dates=True,          # parse date/datetime columns automatically
    null_values=["", "NA", "N/A", "null"],
    schema_overrides={"customer_id": pl.Utf8, "amount": pl.Float64},
    infer_schema_length=10_000,    # rows scanned to infer dtypes (None = full scan, slower but safe)
    ignore_errors=False,
)

# Writing
df.write_csv("out.csv")
```

If a CSV has messy/inconsistent typing, set `infer_schema_length=None` (scans the whole file) or pass explicit `schema_overrides` — silently wrong inferred types are the #1 source of “why is my sum wrong” bugs.

3.3 Parquet (preferred for anything beyond quick scripts)

Parquet is columnar, compressed, typed, and much faster to scan/filter than CSV. Prefer it as your working format for any nontrivial pipeline.

```
df.write_parquet("out.parquet", compression="zstd")
lf = pl.scan_parquet("out.parquet")

# Partitioned datasets (directory of parquet files, e.g. hive-partitioned by date)
lf = pl.scan_parquet("data/year=*/month=/*.parquet", hive_partitioning=True)
```

3.4 Excel

```
df = pl.read_excel("workbook.xlsx", sheet_name="Q3")
df.write_excel("report.xlsx", worksheet="Summary") # needs xlswriter
```

3.5 Databases

```

# Read via a SQL connection string (uses connectorx or adbc under the hood)
df = pl.read_database_uri(
    query="SELECT * FROM sales WHERE order_date >= '2026-01-01'",
    uri="postgresql://user:pass@host:5432/dbname",
)

# Or with an existing DBAPI/SQLAlchemy connection
df = pl.read_database(query="SELECT * FROM sales", connection=my_connection)

# Write back
df.write_database("sales_summary", connection="postgresql://user:pass@host/db", if_table_exists="replace")

```

3.6 JSON / NDJSON

```

df = pl.read_json("records.json")          # a JSON array of objects
df = pl.read_ndjson("events.ndjson")       # newline-delimited JSON (one record per line)
df.write_ndjson("out.ndjson")

```

3.7 In-memory construction

```

df = pl.DataFrame({
    "name": ["Ana", "Bo", "Cy"],
    "age": [23, 31, 42],
    "score": [88.5, 92.1, 79.0],
})

# From list of dicts / rows
df = pl.DataFrame([{"a": 1, "b": 2}, {"a": 3, "b": 4}])
df = pl.from_records([(1, "x"), (2, "y")], schema=["id", "label"])

# From pandas / numpy / arrow (interop)
df = pl.from_pandas(pandas_df)
df = pl.from_numpy(np_array, schema=["a", "b", "c"])
df = pl.from_arrow(arrow_table)

# Back out
pandas_df = df.to_pandas()
np_array = df.to_numpy()
arrow_tbl = df.to_arrow()
records = df.to_dicts()          # list[dict], handy for JSON APIs

```

4. Selecting, Filtering, and Sorting

4.1 Column selection

```
df.select("a", "b")                # by name
df.select(pl.col("a"), pl.col("b")) # equivalent, expression form
df.select(pl.col("^sales_.*$"))     # regex pattern match on names
df.select(pl.col(pl.Float64))       # by dtype
df.select(pl.all())                 # all columns
df.select(pl.all().exclude("id", "created_at"))
df.select(pl.first(), pl.last())    # first/last columns
df[:, ["a", "b"]]                  # indexing shorthand (eager only)
df.drop("id")                       # drop columns
```

`cs` (the `polars.selectors` module) gives more expressive, composable column selection — very useful for “operate on all numeric columns” style requests:

```
import polars.selectors as cs

df.select(cs.numeric())              # all int/float columns
df.select(cs.string())
df.select(cs.datetime())
df.select(cs.numeric() - cs.by_name("id")) # numeric columns except id
df.with_columns(cs.numeric().fill_null(0))
df.select(cs.contains("amount"))
df.select(cs.starts_with("is_"))
```

4.2 Row filtering

```
df.filter(pl.col("age") > 30)
df.filter((pl.col("age") > 30) & (pl.col("country") == "DE")) # use & | ~, not and/or/not
df.filter(pl.col("category").is_in(["A", "B"]))
df.filter(pl.col("email").is_null())
df.filter(pl.col("name").str.contains("(?i)smith")) # regex, (?i) = case-insensitive
df.filter(pl.col("order_date").is_between(pl.date(2020, 1, 1), pl.date(2020, 3, 31)))
df.filter(pl.col("revenue").is_not_nan() & pl.col("revenue").is_finite())

# Row slicing
df.head(10)
df.tail(10)
df.slice(offset=100, length=50)
df.sample(n=100, seed=42)
df.sample(fraction=0.1, seed=42)
```

Important: use `&`, `|`, `~` for combining boolean expressions, always with parentheses around each condition — Python’s `and` / `or` / `not` and operator precedence do not work correctly with polars expressions.

4.3 Sorting

```
df.sort("revenue")
df.sort("revenue", descending=True)
df.sort(["region", "revenue"], descending=[False, True])
df.top_k(10, by="revenue") # fast top-N without a full sort
df.bottom_k(10, by="revenue")
```

5. Expressions Deep Dive

Expressions are the heart of polars. Learning to think in expressions (instead of loops) is the single most important skill for generating good polars code.

5.1 Building blocks

```
pl.col("x")           # reference a column
pl.col("x", "y", "z") # reference multiple columns at once
pl.lit(5)             # a literal scalar value as an expression
pl.col("x").alias("renamed_x") # rename result
pl.all()              # every column
pl.exclude("id")       # every column except id
```

5.2 Arithmetic and comparisons work directly on expressions

```
pl.col("price") * pl.col("qty")
pl.col("revenue") - pl.col("cost")
pl.col("score") / 100
pl.col("age") >= 18
pl.col("value").pow(2)
pl.col("x").sqrt()
pl.col("x").log()
pl.col("x").abs()
pl.col("x").clip(0, 100) # clamp to a range
```

5.3 `when / then / otherwise` — polars' if/else

This is the standard way to do conditional logic, equivalent to `CASE WHEN` in SQL or `np.select` in numpy.

```
df.with_columns(
    pl.when(pl.col("score") >= 90).then(pl.lit("A"))
      .when(pl.col("score") >= 80).then(pl.lit("B"))
      .when(pl.col("score") >= 70).then(pl.lit("C"))
      .otherwise(pl.lit("F"))
      .alias("grade")
)

# Conditional numeric computation
df.with_columns(
    pl.when(pl.col("channel") == "paid").then(pl.col("revenue") * 0.7)
      .otherwise(pl.col("revenue"))
      .alias("net_revenue")
)
```

5.4 Applying an expression to many columns at once

```
df.with_columns(pl.col("q1", "q2", "q3", "q4").fill_null(0))
df.with_columns((pl.col(pl.Float64) * 1.08).name.suffix("_with_tax"))
df.with_columns(pl.col("a", "b", "c").round(2))

# name.prefix / name.suffix / name.map for renaming batches
df.select(pl.all().name.suffix("_raw"))
```

5.5 Aggregation expressions (used inside `.agg()` or `select` on a whole df)

```

pl.col("revenue").sum()
pl.col("revenue").mean()
pl.col("revenue").median()
pl.col("revenue").std()
pl.col("revenue").var()
pl.col("revenue").min()
pl.col("revenue").max()
pl.col("revenue").quantile(0.9)
pl.col("customer_id").n_unique()
pl.col("order_id").count()      # non-null count
pl.len()                       # row count (preferred over deprecated pl.count())
pl.col("revenue").first()
pl.col("revenue").last()
pl.col("category").mode()
pl.col("revenue").sum().alias("total_revenue")

```

5.6 Combining multiple expressions cleanly

Build a list of expressions and unpack it — this is the idiomatic way to keep complex `select / agg` calls readable and reusable:

```

kpi_exprs = [
    pl.col("revenue").sum().alias("total_revenue"),
    pl.col("order_id").n_unique().alias("orders"),
    (pl.col("revenue").sum() / pl.col("order_id").n_unique()).alias("aov"),
    pl.col("customer_id").n_unique().alias("unique_customers"),
]
summary = df.group_by("region").agg(kpi_exprs)

```

6. Transformations: with_columns, casting, struct/list

6.1 Adding and replacing columns

```
df = df.with_columns(
    (pl.col("revenue") - pl.col("cost")).alias("profit"),
    (pl.col("profit") / pl.col("revenue")).alias("margin"),
)
# Note: expressions in the same with_columns call CAN reference each other
# left to right is not guaranteed across all polars versions for cross-refs –
# safest pattern is chaining separate with_columns calls when one depends on another:
df = (
    df.with_columns((pl.col("revenue") - pl.col("cost")).alias("profit"))
    .with_columns((pl.col("profit") / pl.col("revenue")).alias("margin"))
)
```

6.2 Casting types

```
df.with_columns(pl.col("id").cast(pl.Utf8))
df.with_columns(pl.col("price").cast(pl.Float64))
df.with_columns(pl.col("flag").cast(pl.Boolean))
df.with_columns(pl.col("order_date").cast(pl.Date))
df.with_columns(pl.col("category").cast(pl.Categorical)) # memory-efficient for low-cardinality strings
df.with_columns(pl.col("amount").cast(pl.Int64, strict=False)) # non-strict: invalid -> null instead of error
```

6.3 Renaming, reordering, dropping

```
df.rename({"old_name": "new_name"})
df.select(["b", "a", "c"]) # reorder by explicit select
df.drop("unnneeded_col")
df.drop_nulls() # drop rows with any null
df.drop_nulls(subset=["email"]) # only consider this column
df.unique(subset=["customer_id"], keep="first")
df.unique() # drop exact duplicate rows
```

6.4 Structs (nested columns) and Lists

Polars supports nested types natively — useful for JSON-like or grouped data.

```
# Struct: pack multiple columns into one nested column
df.with_columns(pl.struct(["lat", "lon"]).alias("coords"))
df.select(pl.col("coords").struct.field("lat"))
df.unnest("coords") # explode struct fields back into columns

# List: one cell holds a list of values (e.g. after group_by(...).agg(pl.col(...)))
grouped = df.group_by("customer_id").agg(pl.col("order_id")) # order_id becomes List[i64]
grouped.with_columns(pl.col("order_id").list.len().alias("n_orders"))
grouped.explode("order_id") # one row per list element (inverse of group+agg-list)
grouped.with_columns(pl.col("order_id").list.first())
grouped.with_columns(pl.col("order_id").list.contains(12345))
```

7. String, Date/Time, and List Operations

All string operations live under the `.str` namespace, all date/time under `.dt`, all list ops under `.list`. This namespacing is consistent and important to remember.

7.1 String operations (`.str`)

```
pl.col("name").str.to_uppercase()
pl.col("name").str.to_lowercase()
pl.col("name").str.strip_chars()           # trim whitespace both ends
pl.col("name").str.len_chars()             # character length
pl.col("name").str.contains("foo")         # regex by default
pl.col("name").str.contains("foo", literal=True)
pl.col("name").str.starts_with("Mr")
pl.col("name").str.ends_with("Jr")
pl.col("name").str.replace("old", "new")   # first match
pl.col("name").str.replace_all("old", "new") # all matches
pl.col("csv_field").str.split(",")         # -> List[str]
pl.col("full_name").str.split_exact(" ", 1).struct.rename_fields(["first", "last"])
pl.col("code").str.slice(0, 3)
pl.col("code").str.zfill(4)                # zero-pad
pl.col("price_str").str.extract(r"(\d+\.\d+)", 1).cast(pl.Float64) # regex extract + cast
pl.concat_str([pl.col("first"), pl.col("last")], separator=" ").alias("full_name")
```

7.2 Date/time operations (`.dt`)

```
pl.col("order_date").dt.year()
pl.col("order_date").dt.month()
pl.col("order_date").dt.day()
pl.col("order_date").dt.weekday()         # 1=Monday .. 7=Sunday
pl.col("order_date").dt.week()
pl.col("order_date").dt.quarter()
pl.col("order_ts").dt.date()               # datetime -> date
pl.col("order_ts").dt.truncate("1mo")     # floor to start of month
pl.col("order_ts").dt.truncate("1w")
pl.col("order_ts").dt.strftime("%Y-%m-%d")
pl.col("date_str").str.to_date("%Y-%m-%d")
pl.col("ts_str").str.to_datetime("%Y-%m-%d %H:%M:%S", time_zone="UTC")

# Durations and date arithmetic
(pl.col("ship_date") - pl.col("order_date")).dt.total_days().alias("days_to_ship")
pl.col("order_date") + pl.duration(days=30)
pl.date_range(pl.date(2020, 1, 1), pl.date(2020, 12, 31), interval="1d", eager=True)
```

7.3 List operations (`.list`)

```
pl.col("tags").list.len()
pl.col("tags").list.contains("vip")
pl.col("scores").list.mean()
pl.col("scores").list.max()
pl.col("tags").list.unique()
pl.col("tags").list.sort()
pl.col("tags").list.get(0)                 # first element (or null if 00B)
pl.col("tags").list.join(", ")            # list[str] -> str
```

8. Aggregations and `group_by`

8.1 Basic `group_by` / `agg`

```
df.group_by("region").agg(
    pl.col("revenue").sum().alias("total_revenue"),
    pl.col("order_id").n_unique().alias("orders"),
    pl.col("customer_id").n_unique().alias("customers"),
    pl.col("revenue").mean().round(2).alias("avg_order_value"),
)

# Multiple grouping keys
df.group_by(["region", "product_category"]).agg(pl.col("revenue").sum())

# maintain_order=True keeps group order stable (matches first-seen order); costs a bit of speed
df.group_by("region", maintain_order=True).agg(pl.col("revenue").sum())
```

8.2 Multiple aggregations, and getting a list of rows per group

```
df.group_by("customer_id").agg(
    pl.col("order_id"), # -> List[i64], one list per customer
    pl.col("revenue").sum().alias("lifetime_value"),
    pl.col("order_date").min().alias("first_order"),
    pl.col("order_date").max().alias("last_order"),
    pl.len().alias("n_orders"),
)
```

8.3 `group_by_dynamic` — time-bucketed aggregation (downsampling)

The go-to tool for “revenue by week/month” or any resample-style report. Requires the data sorted by the time column.

```
(
    df.sort("order_ts")
    .group_by_dynamic("order_ts", every="1mo")
    .agg(
        pl.col("revenue").sum().alias("monthly_revenue"),
        pl.col("order_id").n_unique().alias("orders"),
    )
)

# With a grouping key in addition to time (per-region monthly trend)
(
    df.sort("order_ts")
    .group_by_dynamic("order_ts", every="1mo", group_by="region")
    .agg(pl.col("revenue").sum())
)
```

Common `every` / period strings: "1d", "1w", "1mo", "1q", "1y", "3mo", "15m" (minutes), "1h".

8.4 Rolling aggregations over time (`rolling` / `group_by_rolling` equivalent via `.over()` + `window`)

```
# Rolling 7-day sum per row, time-based window
(
    df.sort("order_ts")
    .rolling("order_ts", period="7d")
    .agg(pl.col("revenue").sum().alias("revenue_7d"))
)

# Simple rolling stats as new columns (row-count based windows)
df.with_columns(
    pl.col("revenue").rolling_mean(window_size=7).alias("revenue_ma7"),
    pl.col("revenue").rolling_sum(window_size=7).alias("revenue_sum7"),
    pl.col("revenue").rolling_std(window_size=7).alias("revenue_std7"),
)
```

8.5 Pivoting an aggregation into wide format directly

```
df.pivot(  
    values="revenue",  
    index="region",  
    on="product_category",  
    aggregate_function="sum",  
)
```

9. Window Functions (`.over()`)

`.over()` computes an aggregate or ranking **per group while keeping the original row count** — equivalent to SQL `OVER (PARTITION BY ...)`. This is one of the most powerful and underused polars features; use it instead of “group_by then join back” whenever you need a per-group value alongside the original rows.

```
# Add each customer's total spend as a column on every one of their order rows
df.with_columns(pl.col("revenue").sum().over("customer_id").alias("customer_ltv"))

# Share of category total that this row represents
df.with_columns(
    (pl.col("revenue") / pl.col("revenue").sum().over("category")).alias("pct_of_category")
)

# Rank within group
df.with_columns(pl.col("revenue").rank(descending=True).over("region").alias("rank_in_region"))

# Row number within group, e.g. "each customer's Nth order"
df.with_columns(pl.int_range(pl.len()).over("customer_id").alias("order_seq"))
# or, more directly:
df.with_columns(pl.col("order_date").rank(method="ordinal").over("customer_id").alias("order_seq"))

# Difference from previous row within a group (period-over-period, per entity)
df = df.sort(["customer_id", "order_date"])
df = df.with_columns(
    (pl.col("revenue") - pl.col("revenue").shift(1).over("customer_id")).alias("revenue_change")
)

# Multiple partition keys
df.with_columns(pl.col("revenue").mean().over(["region", "channel"]).alias("segment_avg"))

# Flag first/last row in each group
df.with_columns(
    (pl.int_range(pl.len()).over("customer_id") == 0).alias("is_first_order")
)
```

`.over()` combined with `when/then` is the standard pattern for “flag outliers relative to their own group”:

```
df.with_columns(
    pl.when(pl.col("revenue") > pl.col("revenue").mean().over("region") + 3 * pl.col("revenue").std().over("region"))
        .then(pl.lit(True)).otherwise(pl.lit(False))
        .alias("is_outlier_in_region")
)
```

10. Joins, Concatenation, and Reshaping

10.1 Joins

```
orders.join(customers, on="customer_id", how="left")
orders.join(customers, on="customer_id", how="inner")
orders.join(customers, on="customer_id", how="full")      # full outer; was "outer" pre-1.0
orders.join(customers, on="customer_id", how="semi")      # keep left rows that DO match (no right cols)
orders.join(customers, on="customer_id", how="anti")      # keep left rows that DON'T match
orders.join(customers, left_on="cust_id", right_on="id", how="left")

# Multi-key join
orders.join(prices, on=["product_id", "effective_date"], how="left")

# As-of join: match each row to the nearest prior row in another table by a sorted key
# (classic use: attach the most recent price/exchange-rate as of the transaction date)
trades.sort("trade_time").join_asof(
    rates.sort("rate_time"),
    left_on="trade_time", right_on="rate_time",
    strategy="backward",
)

# Cross join (cartesian product) - e.g. build every date x every store
dates.join(stores, how="cross")
```

`how="full"` produces `_right`-suffixed duplicate key columns; use `coalesce=True` to merge them automatically:

```
a.join(b, on="id", how="full", coalesce=True)
```

10.2 Concatenation

```
pl.concat([df1, df2])      # stack rows (union), same schema
pl.concat([df1, df2], how="diagonal") # union with differing/missing columns -> nulls filled in
pl.concat([df1, df2], how="horizontal") # side-by-side column concat (row-count must match)
pl.concat([df1, df2], how="vertical_relaxed") # relaxes dtype mismatches, casts as needed
```

10.3 Reshaping: pivot (long -> wide) and unpivot (wide -> long)

```
# Long -> wide
wide = df.pivot(values="revenue", index="month", on="region", aggregate_function="sum")

# Wide -> long (formerly "melt")
long = wide.unpivot(index="month", on=["EMEA", "NA", "APAC"], variable_name="region", value_name="revenue")
```

11. Missing Data Handling

```
df.null_count()                                # nulls per column
df.filter(pl.col("email").is_null())
df.filter(pl.col("email").is_not_null())
df.with_columns(pl.col("revenue").fill_null(0))
df.with_columns(pl.col("revenue").fill_null(strategy="mean"))
df.with_columns(pl.col("revenue").fill_null(strategy="forward")) # carry last valid value forward
df.with_columns(pl.col("category").fill_null("Unknown"))
df.with_columns(pl.col("value").interpolate()) # linear interpolation across nulls (time series gaps)
df.drop_nulls()
df.drop_nulls(subset=["customer_id", "order_date"])

# NaN vs null are different in polars (NaN is a float value; null is "missing")
df.with_columns(pl.col("x").fill_nan(None)) # convert NaN to null first if you want fill_null to catch it
pl.col("x").is_nan()
pl.col("x").is_finite()
```

12. Performance: Lazy Execution, Streaming, Query Plans

12.1 Always prefer lazy for pipelines

```
lf = pl.scan_parquet("events/*.parquet")
q = (
    lf.filter(pl.col("event_date") >= pl.date(2026, 1, 1))
      .select("user_id", "event_type", "revenue")
      .group_by("user_id")
      .agg(pl.col("revenue").sum())
)
print(q.explain())      # see the optimized plan before running
df = q.collect()        # execute once, optimized end-to-end
```

12.2 Streaming for datasets bigger than RAM

```
df = q.collect(engine="streaming")  # processes in chunks instead of materializing everything
q.sink_parquet("result.parquet")    # write results directly, streaming, without full materialization
```

12.3 Query plan inspection

```
q.explain()                # optimized logical plan
q.explain(optimized=False) # naive/unoptimized plan, for comparison
q.show_graph()             # visual plan (needs graphviz), useful in notebooks
```

12.4 General performance habits

- Read only the columns you need: `select` early, or pass `columns=[...]` to `read_parquet` / `scan_parquet`.
- Filter as early as possible in a lazy chain — the optimizer will push it down into the file scan when possible, but writing it early makes intent clear and works even in edge cases the optimizer can't rewrite.
- Prefer `pl.Categorical` for low-cardinality string columns used in `group_by` / joins — much faster comparisons and less memory than `Utf8`.
- Avoid `.apply()` / `.map_elements()` — they run row-by-row in Python and disable most optimizations. Use them only as a last resort for logic with no expression equivalent, and prefer `return_dtype=` to avoid schema-inference overhead when you must.
- Use `.collect(engine="streaming")` or `sink_*` for pipelines over data that doesn't fit in memory.
- `df.estimated_size("mb")` to check a DataFrame's memory footprint.

13. Statistical Analysis with Polars

Polars covers descriptive statistics natively; for inferential statistics (t-tests, ANOVA, regression) hand off small extracted arrays to `scipy / numpy / statsmodels`, since polars itself does not implement hypothesis tests.

13.1 Descriptive statistics

```
df.describe()                                # count, mean, std, min, quartiles, max per numeric column

df.select(
    pl.col("revenue").mean().alias("mean"),
    pl.col("revenue").median().alias("median"),
    pl.col("revenue").std().alias("std_dev"),
    pl.col("revenue").var().alias("variance"),
    pl.col("revenue").min().alias("min"),
    pl.col("revenue").max().alias("max"),
    pl.col("revenue").quantile(0.25).alias("p25"),
    pl.col("revenue").quantile(0.75).alias("p75"),
    pl.col("revenue").skew().alias("skewness"),
    pl.col("revenue").kurtosis().alias("kurtosis"),
)
```

13.2 Correlation

```
# Pairwise correlation of two columns
df.select(pl.corr("price", "units_sold"))

# Full correlation matrix across numeric columns (compute pairwise then pivot, or use numpy)
import numpy as np
numeric_cols = [c for c, dt in zip(df.columns, df.dtypes) if dt.is_numeric()]
corr_matrix = np.corrcoef(df.select(numeric_cols).to_numpy(), rowvar=False)
corr_df = pl.DataFrame(corr_matrix, schema=numeric_cols).with_columns(pl.Series("metric", numeric_cols))
```

13.3 Outlier detection

```
# IQR method
q1 = df.select(pl.col("revenue").quantile(0.25)).item()
q3 = df.select(pl.col("revenue").quantile(0.75)).item()
iqr = q3 - q1
lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr
outliers = df.filter((pl.col("revenue") < lower) | (pl.col("revenue") > upper))

# Z-score method
df = df.with_columns(
    ((pl.col("revenue") - pl.col("revenue").mean()) / pl.col("revenue").std()).alias("z_score")
)
outliers = df.filter(pl.col("z_score").abs() > 3)
```

13.4 Trend and period-over-period analysis

```
monthly = (
    df.sort("order_ts")
    .group_by_dynamic("order_ts", every="1mo")
    .agg(pl.col("revenue").sum().alias("revenue"))
    .with_columns(
        pl.col("revenue").pct_change().alias("mom_growth"),
        (pl.col("revenue") / pl.col("revenue").shift(12) - 1).alias("yoy_growth"),
        pl.col("revenue").rolling_mean(window_size=3).alias("revenue_3mo_avg"),
    )
)
```

13.5 Hypothesis testing and regression (hand off to scipy/statsmodels/numpy)

```

import numpy as np
from scipy import stats

group_a = df.filter(pl.col("variant") == "A")["conversion_rate"].to_numpy()
group_b = df.filter(pl.col("variant") == "B")["conversion_rate"].to_numpy()
t_stat, p_value = stats.ttest_ind(group_a, group_b, equal_var=False)

# Simple linear regression via numpy polyfit
x = df["marketing_spend"].to_numpy()
y = df["revenue"].to_numpy()
slope, intercept = np.polyfit(x, y, 1)

# Chi-square test of independence on a crosstab
crosstab = df.pivot(values="count", index="segment", on="converted", aggregate_function="sum").fill_null(0)
chi2, p, dof, expected = stats.chi2_contingency(crosstab.drop("segment").to_numpy())

```

13.6 Cohort / retention analysis pattern

```

cohorts = (
    df.with_columns(pl.col("signup_date").dt.truncate("1mo").alias("cohort_month"))
    .with_columns(
        ((pl.col("order_date").dt.truncate("1mo") - pl.col("cohort_month")).dt.total_days() / 30)
        .round(0).cast(pl.Int32).alias("months_since_signup")
    )
    .group_by(["cohort_month", "months_since_signup"])
    .agg(pl.col("customer_id").n_unique().alias("active_customers"))
    .sort(["cohort_month", "months_since_signup"])
)
cohort_sizes = cohorts.filter(pl.col("months_since_signup") == 0).select("cohort_month",
pl.col("active_customers").alias("cohort_size"))
retention = (
    cohorts.join(cohort_sizes, on="cohort_month")
    .with_columns((pl.col("active_customers") / pl.col("cohort_size")).alias("retention_rate"))
    .pivot(values="retention_rate", index="cohort_month", on="months_since_signup")
)

```

14. Data Consulting Workflows

These are template sequences for common client-style requests. Follow the shape; adapt column names.

14.1 Workflow: Profile a new/unfamiliar dataset

Always do this before any analysis on data you haven't seen.

```
print(df.shape)
print(df.schema)                # column name -> dtype
print(df.head(10))
print(df.describe())
print(df.null_count())
for col in df.columns:
    n_unique = df.select(pl.col(col).n_unique()).item()
    print(f"{col}: {n_unique} unique values")
# Look for duplicate rows / duplicate keys
print(df.is_duplicated().sum())
print(df.group_by("id_column").len().filter(pl.col("len") > 1)) # duplicate primary keys
```

Report to the user: row/column count, dtypes, null rates per column, obvious data quality issues (duplicate keys, suspicious constant columns, unparsed date strings, mixed types), and the date range covered if there's a date column.

14.2 Workflow: Diagnose a metric drop / “why did X change”

1. Confirm the change is real: aggregate the metric at the reported grain (day/week) and re-derive the before/after numbers independently.
2. Decompose by every available dimension (region, channel, product, segment, customer cohort) to find where the change concentrates — a metric that drops uniformly everywhere has a different cause than one that drops in one segment.
3. Check for confounds: seasonality (compare to the same period last year), a definition/pipeline change (did null rates or row counts shift?), mix-shift (did the population composition change even if per-segment rates didn't?).

```
current = df.filter(pl.col("week") == current_week)
prior = df.filter(pl.col("week") == prior_week)

def kpi(frame):
    return frame.select(
        pl.col("revenue").sum().alias("revenue"),
        pl.col("order_id").n_unique().alias("orders"),
        (pl.col("revenue").sum() / pl.col("order_id").n_unique()).alias("aov"),
    )

print("current:", kpi(current))
print("prior:", kpi(prior))

# Decomposition by segment: which segments moved the most?
decomp = (
    df.filter(pl.col("week").is_in([current_week, prior_week]))
    .group_by(["segment", "week"])
    .agg(pl.col("revenue").sum())
    .pivot(values="revenue", index="segment", on="week")
    .with_columns((pl.col(str(current_week)) - pl.col(str(prior_week))).alias("delta"))
    .sort("delta")
)

# Mix-shift check: did segment sizes change even if per-segment behavior didn't?
mix = (
    df.filter(pl.col("week").is_in([current_week, prior_week]))
    .group_by(["segment", "week"])
    .agg(pl.col("order_id").n_unique().alias("orders"))
    .with_columns((pl.col("orders") / pl.col("orders").sum().over("week")).alias("share_of_orders"))
)
```

14.3 Workflow: Segment / cohort comparison

```

segment_summary = (
    df.group_by("segment")
      .agg(
        pl.col("customer_id").n_unique().alias("customers"),
        pl.col("revenue").sum().alias("total_revenue"),
        pl.col("revenue").mean().alias("avg_revenue_per_order"),
        (pl.col("revenue").sum() / pl.col("customer_id").n_unique()).alias("revenue_per_customer"),
      )
      .with_columns((pl.col("total_revenue") / pl.col("total_revenue").sum() * 100).round(1).alias("pct_of_total"))
      .sort("total_revenue", descending=True)
)

```

14.4 Workflow: Build a KPI / executive summary table

```

kpis = df.select(
    pl.col("revenue").sum().alias("total_revenue"),
    pl.col("order_id").n_unique().alias("total_orders"),
    pl.col("customer_id").n_unique().alias("total_customers"),
    (pl.col("revenue").sum() / pl.col("order_id").n_unique()).round(1).alias("avg_order_value"),
    (pl.col("revenue").sum() / pl.col("customer_id").n_unique()).round(1).alias("revenue_per_customer"),
)
# Then present as a short narrative: "Total revenue was $X across N orders from M customers,
# an average order value of $Y – up/down Z% vs the prior period."

```

14.5 Workflow: Data quality / anomaly sweep before trusting a number

```

issues = []
if df.select(pl.col("order_date").is_null().sum()).item() > 0:
    issues.append("missing order dates")
if df.filter(pl.col("revenue") < 0).height > 0:
    issues.append("negative revenue rows")
dup_keys = df.group_by("order_id").len().filter(pl.col("len") > 1)
if dup_keys.height > 0:
    issues.append(f"{dup_keys.height} duplicated order_id values")
future_dates = df.filter(pl.col("order_date") > pl.date.today())
if future_dates.height > 0:
    issues.append(f"{future_dates.height} rows with future order dates")
print("Data quality issues found:", issues or "none")

```

14.6 How to present results (report structure)

When asked for “a report” or “an analysis,” structure the written output as: (1) headline finding in one sentence, (2) supporting numbers in a small table, (3) one chart if it clarifies a trend or comparison, (4) caveats/data-quality notes, (5) a recommended next step or decision the numbers imply. Avoid dumping raw `df.describe()` output as the final answer — synthesize it into plain-language sentences with the specific numbers embedded.

15. Charting and Visualization

15.1 Quick native charts: `df.plot.*` (Altair backend)

Since polars 1.x, `DataFrame.plot` is backed by Altair (requires `pip install altair`). It returns an Altair `Chart` object — good for quick, interactive exploration, especially in notebooks. This produces HTML/JSON, not a static image directly.

```
import polars as pl

chart = df.plot.line(x="order_date", y="revenue")
chart = df.plot.bar(x="region", y="total_revenue")
chart = df.plot.scatter(x="marketing_spend", y="revenue", color="region")
chart = df.plot.point(x="age", y="score")
chart.save("chart.html") # or chart.save("chart.png") if vl-convert-python is installed
```

15.2 Matplotlib (best for static, print-ready report charts)

Convert the columns you need to numpy/pandas and plot with matplotlib for full control over static image output (PNG/PDF for a report).

```
import matplotlib.pyplot as plt

monthly = df.group_by_dynamic("order_ts", every="1mo").agg(pl.col("revenue").sum())

fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(monthly["order_ts"].to_list(), monthly["revenue"].to_list(), marker="o")
ax.set_title("Monthly Revenue Trend")
ax.set_xlabel("Month")
ax.set_ylabel("Revenue ($)")
ax.grid(alpha=0.3)
fig.tight_layout()
fig.savefig("monthly_revenue.png", dpi=150)
plt.close(fig)
```

```
# Bar chart comparing categories
summary = df.group_by("region").agg(pl.col("revenue").sum()).sort("revenue", descending=True)
fig, ax = plt.subplots(figsize=(8, 5))
ax.bar(summary["region"].to_list(), summary["revenue"].to_list(), color="#4C72B0")
ax.set_title("Revenue by Region")
ax.set_ylabel("Revenue ($)")
plt.xticks(rotation=30, ha="right")
fig.tight_layout()
fig.savefig("revenue_by_region.png", dpi=150)
```

```
# Multi-series line chart (one line per group) – pivot wide first
wide = df.group_by_dynamic("order_ts", every="1mo", group_by="region").agg(pl.col("revenue").sum()) \
    .pivot(values="revenue", index="order_ts", on="region")
fig, ax = plt.subplots(figsize=(10, 5))
for region_col in wide.columns[1:]:
    ax.plot(wide["order_ts"].to_list(), wide[region_col].to_list(), label=region_col, marker="o")
ax.legend()
ax.set_title("Revenue Trend by Region")
fig.tight_layout()
fig.savefig("revenue_by_region_trend.png", dpi=150)
```

15.3 Plotly (best for interactive dashboards / HTML deliverables)

```
import plotly.express as px

pdf = df.group_by("category").agg(pl.col("revenue").sum()).sort("revenue", descending=True).to_pandas()
fig = px.bar(pdf, x="category", y="revenue", title="Revenue by Category")
fig.write_html("revenue_by_category.html")
fig.write_image("revenue_by_category.png") # requires kaleido
```

15.4 Chart-type selection guide

Question being answered	Chart type
How does a metric trend over time?	Line chart
How do categories compare on one metric?	Bar chart (sorted, horizontal if many categories)
How is a metric distributed?	Histogram / box plot
Is there a relationship between two numeric variables?	Scatter plot
How does a whole break into parts, and does that mix matter more than the total?	Stacked bar / 100% stacked bar (avoid pie charts beyond 3-4 slices)
How does a metric move over time across several groups?	Multi-line chart or small multiples
Where are entities located / regional comparison?	Choropleth / geo map (plotly)

Defaults for any chart produced for a report: always title the chart, label both axes with units, sort categorical bars by value rather than alphabetically unless there's a natural order (e.g. months), and never use a pie chart with more than ~5 slices.

16. End-to-End Worked Case Study

Scenario: A client asks — “Revenue looked flat last quarter versus last year. What’s going on, and what should we do?” Assume a parquet file `orders.parquet` with columns `order_id`, `customer_id`, `order_date`, `region`, `channel`, `product_category`, `revenue`, `cost`.

```
import polars as pl
import matplotlib.pyplot as plt

lf = pl.scan_parquet("orders.parquet")

# 1. Profile
df_sample = lf.head(5).collect()
schema = lf.collect_schema()
print(schema)

# 2. Define the comparison windows
this_q = (pl.date(2024, 4, 1), pl.date(2024, 6, 30))
last_q_ly = (pl.date(2023, 4, 1), pl.date(2023, 6, 30))

def window_kpis(lf, start, end):
    return (
        lf.filter(pl.col("order_date").is_between(start, end))
        .select(
            pl.col("revenue").sum().alias("revenue"),
            pl.col("order_id").n_unique().alias("orders"),
            pl.col("customer_id").n_unique().alias("customers"),
        )
        .collect()
    )

cur = window_kpis(lf, *this_q)
prior = window_kpis(lf, *last_q_ly)
print("This Q:", cur)
print("Same Q last year:", prior)

# 3. Decompose by region and channel to find where the flatness concentrates
decomp = (
    lf.filter(
        pl.col("order_date").is_between(*this_q) | pl.col("order_date").is_between(*last_q_ly)
    )
    .with_columns(
        pl.when(pl.col("order_date").is_between(*this_q)).then(pl.lit("this_q"))
        .otherwise(pl.lit("last_q_ly")).alias("period")
    )
    .group_by(["region", "channel", "period"])
    .agg(pl.col("revenue").sum())
    .collect()
    .pivot(values="revenue", index=["region", "channel"], on="period")
    .with_columns(((pl.col("this_q") - pl.col("last_q_ly")) / pl.col("last_q_ly") * 100).round(1).alias("yoy_pct"))
    .sort("yoy_pct")
)
print(decomp)

# 4. Monthly trend chart for the narrative
monthly = (
    lf.filter(pl.col("order_date") >= pl.date(2024, 1, 1))
    .sort("order_date")
    .group_by_dynamic("order_date", every="1mo")
    .agg(pl.col("revenue").sum())
    .collect()
)

fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(monthly["order_date"].to_list(), monthly["revenue"].to_list(), marker="o")
ax.set_title("Monthly Revenue, 2024-2026")
ax.set_ylabel("Revenue ($)")
fig.tight_layout()
fig.savefig("monthly_revenue_trend.png", dpi=150)

# 5. Narrative (what the assistant should write back to the user):
# "Total revenue was flat (+0.4% YoY) this quarter, but that hides a split: the paid channel in
# APAC declined 18% YoY while organic in NA grew 22% YoY, offsetting it. The decline in APAC paid
# concentrates in the last 6 weeks and coincides with a drop in unique customers, not order value –
# suggesting an acquisition/traffic issue rather than a pricing or basket-size issue. Recommend
# checking APAC paid channel spend and conversion funnels for that window before drawing broader
# conclusions."
```

17. Common Pitfalls and Anti-Patterns

- **Using `and` / `or` / `not` on expressions.** Use `&`, `|`, `~` with each condition parenthesized: `(pl.col("a") > 1) & (pl.col("b") < 2)`.
- **Looping over `df.iter_rows()` for computation.** This is 10-100x slower than expressions and rarely necessary. Reach for `.over()`, `when/then`, or a `join` instead.
- **Calling `.collect()` repeatedly mid-pipeline.** Build the whole lazy chain, then collect once at the end so the optimizer sees the full plan.
- **Comparing floats for exact equality.** Use `.is_close()` / a tolerance check, not `==`, for computed floating point values.
- **Forgetting `is_between` is inclusive by default.** Pass `closed="left" / "right" / "none"` if you need an exclusive bound.
- **Assuming `group_by` preserves input row order.** Pass `maintain_order=True` explicitly if downstream code depends on group order.
- **Using `pl.count()`.** Deprecated; use `pl.len()` for row counts.
- **Mixing pandas and polars idioms.** `df["col"]` on a polars DataFrame returns a Series (fine for a single column), but polars has no `.loc` / `.iloc`-style label indexing — use `.filter()` + `.select()`.
- **Ignoring dtype inference on CSV read.** A numeric-looking column read as `Utf8` due to a stray non-numeric value will silently break `.sum()` / `.mean()`. Check `df.schema` early and set `schema_overrides` when needed.
- **Doing a `join` before checking for duplicate keys on the right side.** A one-to-many join where you expected one-to-one silently multiplies rows; verify with `right_df.group_by(key).len().filter(pl.col("len") > 1)` first.

18. Quick-Reference Cheat Sheet

```
# --- I/O ---
pl.read_csv(path) / pl.scan_csv(path)
pl.read_parquet(path) / pl.scan_parquet(path)
df.write_csv(path) / df.write_parquet(path)

# --- Inspect ---
df.shape, df.schema, df.columns, df.dtypes
df.head(), df.describe(), df.null_count()

# --- Select / filter / sort ---
df.select(pl.col("a"), pl.col("b"))
df.filter((pl.col("a") > 1) & (pl.col("b").is_in([...])))
df.sort("a", descending=True)

# --- Transform ---
df.with_columns((pl.col("a") * pl.col("b")).alias("c"))
pl.when(cond).then(x).otherwise(y)
df.with_columns(pl.col("x").cast(pl.Float64))

# --- Aggregate ---
df.group_by("key").agg(pl.col("value").sum().alias("total"))
df.group_by_dynamic("ts", every="1mo").agg(...)

# --- Window ---
df.with_columns(pl.col("value").sum().over("key").alias("group_total"))

# --- Join / reshape ---
a.join(b, on="key", how="left")
df.pivot(values="v", index="i", on="c", aggregate_function="sum")
df.unpivot(index="i", on=[...])

# --- Stats ---
pl.corr("a", "b")
pl.col("a").quantile(0.9)
pl.col("a").rolling_mean(window_size=7)

# --- Charts ---
df.plot.line(x="a", y="b") # Altair, quick/interactive
plt.plot(df["a"].to_list(), df["b"].to_list()) # matplotlib, static/report
px.bar(df.to_pandas(), x="a", y="b") # plotly, interactive HTML

# --- Namespaces to remember ---
.str.* string ops
.dt.* date/time ops
.list.* list ops
.struct.* nested struct ops
.cat.* categorical ops
```

End of reference. When in doubt: think in expressions, stay lazy until the last step, and always print `.schema / .describe()` before trusting a number.